

RL105: Reinforcement Fine-Tuning and RLHF

How Human Feedback Shapes Modern Language Models

Colby Wang @ SparkCraft

From RL Agents to Language Models

- In previous lectures, we studied RL agents acting in environments.
- Today, the “agent” is a language model.
- The action is not moving left or right.
- The action is generating tokens.

LLMs can be treated as policies over text.

Why Fine-Tuning?

- Pretraining teaches a model to predict the next token.
- But next-token prediction alone does not guarantee helpful behavior.
- We want models that are:
 - helpful
 - harmless
 - honest
 - aligned with user intent

Fine-tuning turns raw capability into useful behavior.

Three Stages of Modern LLM Training

① Pretraining

- Learn general language patterns from large-scale text.

② Supervised Fine-Tuning

- Learn from high-quality instruction-response examples.

③ Reinforcement Fine-Tuning / RLHF

- Optimize model behavior using reward signals.

What is Reinforcement Fine-Tuning?

- Reinforcement Fine-Tuning uses reinforcement learning to improve a model after supervised fine-tuning.
- The model generates outputs.
- A reward signal evaluates the outputs.
- The model is updated to generate better responses.

RFT = fine-tuning with rewards.

Beginner:

- The model tries different answers and learns which ones are better.

Engineer:

- We sample responses, score them, and update the policy.

Researcher:

- We optimize expected reward while controlling distribution shift from the reference model.

- RLHF uses human preferences as a training signal.
- Instead of only asking, “what is the next token?”
- we ask:

Which response would humans prefer?

The RLHF Pipeline

- 1 Collect prompts.
- 2 Generate multiple model responses.
- 3 Ask humans to compare or rank responses.
- 4 Train a reward model from preferences.
- 5 Use RL to optimize the language model.

Human preference becomes a reward signal.

Reward Model

- A reward model predicts how good a response is.
- It is trained from human preference data.
- Instead of manually scoring every response forever, we train a model to approximate human judgment.

$$r_{\phi}(x, y)$$

Prompt + response $\rightarrow$ reward score

Preference Modeling

Suppose humans prefer response y_w over response y_l .

A common preference objective is:

$$\mathcal{L}_{RM} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

- y_w : preferred response
- y_l : less preferred response
- Reward model learns to score preferred responses higher

Where Does RL Enter?

- The language model is the policy:

$$\pi_{\theta}(y|x)$$

- The prompt is the state/context.
- The generated response is the action sequence.
- The reward model provides reward.

We optimize the model to generate high-reward responses.

RLHF Objective

A simplified RLHF objective:

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [r_{\phi}(x, y)]$$

But we usually also constrain the model not to drift too far from the reference model:

$$\max_{\theta} \mathbb{E} [r_{\phi}(x, y) - \beta D_{KL}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))]$$

Why KL Penalty?

- If we only maximize reward, the model may exploit the reward model.
- This can lead to reward hacking.
- The KL penalty keeps the model close to the original supervised model.

Improve behavior, but do not drift too far.

- PPO is commonly used because it limits policy updates.
- It prevents the model from changing too much in one step.
- This makes RLHF more stable.

$$L^{CLIP}(\theta) = \mathbb{E} [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

- Actor: the language model generating responses.
- Critic: value model estimating expected reward.
- Reward model: scores generated outputs.

Actor generates. Critic estimates. Reward model judges.

Simplified RLHF loop

```
advantage = reward - value  
updatepolicywithPPO(advantage)updatecritic(reward)
```

Method	Signal	Goal
SFT	Demonstrations	Imitate good answers
RLHF	Human preferences	Align with human judgment
RFT	Task rewards	Improve task performance

Different feedback signals lead to different fine-tuning strategies.

Why RLHF Matters

- Many useful qualities are hard to capture with next-token prediction.
- Examples:
 - helpfulness
 - instruction following
 - safety
 - reasoning preference
 - conversational quality

RLHF helps optimize what humans actually care about.

Limitations of RLHF

- Human preference data can be noisy.
- Reward models can be imperfect.
- Models may exploit reward model weaknesses.
- RL training can be unstable and expensive.
- Alignment is not fully solved by RLHF alone.

RLHF is powerful, but not magic.

Modern Alternatives and Extensions

- Direct Preference Optimization
- Group Relative Policy Optimization
- Constitutional AI
- Reinforcement Fine-Tuning with verifiable rewards
- Tool-use and agentic RL

The field is moving toward cheaper, more stable alignment methods.

Key Takeaways

- LLMs can be viewed as policies over tokens.
- RLHF uses human preference as reward.
- Reward models approximate human judgment.
- PPO optimizes the model while controlling policy drift.
- RFT extends this idea to task-specific reward signals.

Modern LLM alignment is reinforcement learning in disguise.

- RLHF connects reinforcement learning with human values.
- RFT connects reinforcement learning with task-specific improvement.
- Future LLM agents will likely require:
 - better rewards
 - better environments
 - better evaluation
 - better safety mechanisms

The future is not just better models — it is better feedback.